Undertow - final report and your to-do list

Written 12 October 2026, one day before the 13 October deadline.

Where things stand

The product is built and its numbers are checked against Perpl's own API. What is missing falls into two groups: things that need **your accounts, money or time** (listed in Part 3 with exact steps), and a few product gaps I could not close (Part 2). The bot has **not traded on any network**, and there is no public link yet.

Piece	State
Radar: liquidation zones, plain-language warning, cascade simulator	Done. Cascade is a labelled worst-case estimate, not validated on a real liquidation
Trade: order preview (liquidation, zones in your path, fees, funding, slippage)	Done. Does not send orders
Markets, Protocol, Wallets (search, watchlist, compare), Bot tabs	Done. Wallet profit-and-loss statistics are not built
Liquidation-aware order rules	Done as a live check and rule engine. Checked about once a minute, not every block; live cancelling untested
Independent correctness checks against Perpl's REST API	Done; results below
Bot	Paper mode only. Testnet and mainnet need your wallet
Envio indexer	Built and tested end to end, but the public RPCs rate-limit it, so it holds only a small recent window
Logo, README, architecture, deploy guide, video scripts, submission checklist	Done (in docs/)
Code in a local git repository	Committed locally. Not pushed anywhere
Docker image for hosting	Built, run and fetching fresh data locally; hosts untested

Part 1: how the numbers were verified

Each check compares our value with Perpl's own public API (an independent source) or with Perpl's own SDK. They can be rerun with `python -m undertow.verify` or the button on the dashboard's Protocol tab.

Check	Result	Detail
ticker: mark price	PASS	ours 83,837.70 vs Perpl 83,837.70 (0.000% apart)
ticker: open interest	PASS	ours 9.00990 vs Perpl 9.00990 (0.000% apart)
order book: top 10 levels per side	PASS	20/20 price levels present in both, 20/20 identical in size and order count (both read at block 112,542,673)
volume: indexer vs Perpl candles	SKIPPED	indexer window does not yet cover one whole minute
funding: indexer vs Perpl series	SKIPPED	indexer has no funding events yet
formula: our liquidation price vs Perpl's	PASS	693/693 positions match (relative error <= 0.1% or within the 6-decimal print)
own test account: size, entry and liquidation vs Perpl's Positions table	SKIPPED	needs a funded account on Perpl (not available)

Run at 2026-10-11 18:41:07 UTC on live mainnet data.

What the checks found along the way

- **Order book:** my first comparison failed (16 of 20 levels). The cause was a one-block offset: Perpl's REST book labelled block b is the contract state at block b minus 1. With that alignment all 20 levels match in price, size and order count. I did not weaken the test to make it pass; I measured the offset.
- **Funding units:** the funding rate in our data is a fraction per interval (a 4e-5 rate gave a payment of $3.3 per BTC at a price near $83,000, matching Perpl's series), and the interval is 2,580 seconds, **about 43 minutes**, not an hour as I had labelled it. Fixed on the dashboard. **Correction:** the earlier Step 5 PDF says funding is 'about hourly'; that line is superseded by this one.

- **Not done:** 'own test account' (needs a funded Perpl account), and the volume and funding checks against the indexer (the indexer has not covered a whole minute or hour yet because the public RPCs rate-limit it). They are written and will run once the indexer has data.

Mistakes I caught in my own work this round

- The 'safer leverage' suggestion was logically inverted (it looked for a liquidation *below* every zone, which puts every zone in your path). Rewritten so a leverage is safe if your liquidation sits beyond where the modelled cascade from the biggest zone in your path ends. A unit test now covers deep versus thin books.
- The cascade was aimed at a zone's midpoint, which can sit above the liquidation prices of the very positions in it, so nothing triggered. It now aims at the zone's far edge (also fixed on the Radar headline).
- Order slippage was measured against the mark price, giving a nonsense negative number for a buy. It is now measured against the best price in the book.
- The logo first rendered as a black square because the renderer ignores gradients; redone with solid colours.

Part 2: what I could not finish, and why

- **The public Monad RPCs are unreliable from this machine.** rpc.monad.xyz timed out for hours (Windows and Ubuntu); rpc1 and rpc2 work from Windows but time out from Ubuntu, where only rpc-mainnet.monadinfra.com worked; all of them rate-limit. I made the data layer rotate through several endpoints automatically, but your own RPC (for example Alchemy, which is also a hackathon sponsor) is the real fix.
- **The Envio indexer cannot backfill on public RPCs.** It stalls on rate limits after a few hundred blocks. The proper source (HyperSync) needs a free Envio token, which means creating an account, something I cannot do. With the token it can read the full history since launch within minutes.
- **Docker image (updated 12 October):** now tested end to end locally. The fast build (Dockerfile.fast, a prebuilt Perpl tool, no Rust compile) takes about 15 seconds, runs as a normal user, and fetched fresh data inside the container through a fallback RPC. What is still untested is each hosting company's own build and run steps.
- **Bot on testnet:** needs a funded wallet (below). Nothing about order placement, cancelling or fills has run on any chain.
- **Wallet statistics** (win rate, profit factor, streaks, holding time) and trade history are not built. The indexer records only the resting side of each trade and no per-wallet profit and loss, so this needs more indexing work. The Perpl Risk Tool bounty lists these, so that entry is incomplete.
- **Several-days-of-live-bot-activity** for the API bounty cannot be achieved any more: even if a wallet is funded today there are about 1.5 days left.
- **Cascade model validation:** no real Perpl liquidation has appeared in the indexed window, so the model has never been tested against a real one.

Part 3: what you need to do

In priority order. The deadline is **13 October**; submit on the 13th, not the morning of the 14th. Times are rough.

A. Needed to submit at all

#	What	Exactly how	Time
1	**Put the code on GitHub and share it** (Track 1 requires a public repo shared with metropolis@hackathon.monad.xyz)	Create an empty public repository on your GitHub account. In the project folder run: git remote add origin https://github.com/YOU/undertow.git then git push -u origin main. Then Settings, Collaborators, add metropolis@hackathon.monad.xyz. The code is already committed locally and checked for keys; nothing has been pushed.	10 min
2	**Record the four videos**	Scripts, timings and what to show are in docs/VIDEO_SCRIPTS.md: Track 1 demo (3 min), Track 1 pitch (2 min), bot (2 min), dashboard (2 min). Start the dashboard first (command in the README) and wait until 'Data Ns old' is under a minute. Say only what is true today; the checklist lists claims to avoid.	1 to 2 h
3	**Get a stable RPC and put it in the dashboard**	Create a free Alchemy account, make a Monad mainnet app, copy its HTTPS URL. Then in PowerShell: $env:UNDERTOW_RPC="YOUR-URL" before starting the dashboard. This stops the stale-data problems and unlocks the indexer. Alchemy's hackathon credits ($1,000) apply to projects using it.	10 min
4	**Publish a live link** (Track 1 and the Risk Tool bounty want one)	Create an account with any host that runs Docker containers, point it at your GitHub repository, set UNDERTOW_RPC to your Alchemy URL, deploy. Steps and caveats in docs/DEPLOY.md. If you cannot, say in the submission that the demo runs locally and show the video.	30 to 60 min

B. Makes the Envio entry real

#	What	Exactly how	Time
5	**Free Envio API token**	Create a free token at https://envio.dev/app/api-tokens. Put ENVIO_API_TOKEN=... in indexer/.env, remove the two rpc: blocks and set start_block to 54773010 in indexer/config.yaml (instructions in indexer/README.md), then restart the indexer in Ubuntu. Tell me when done and I can rerun the volume and funding checks and fill the Protocol view with full history.	10 min

C. Needed for the Perpl API (bot) bounty

#	What	Exactly how	Time
6	**Ask Perpl's team three questions** (the roadmap left these open)	Does testnet activity count for the bot bounty? Does using the Rust SDK count as using their API? Can one project enter Track 1 and both Perpl bounties? The answers decide whether C is worth doing.	5 min
7	**Fund the testnet wallet**	Send testnet MON for gas (a small amount) and at least 100 test aUSD to this address: 0x64aD42A53Cde61716920E3C79b5656312A11D6B8. Test aUSD comes from Perpl's testnet app (testnet.perpl.xyz) with your own wallet; nobody can mint it freely. Then I can run python -m undertow.testnet fund 100 and the bot on testnet and review the log with you.	10 to 20 min
8	**Decide on mainnet**	Only after a clean testnet run you have reviewed: a separate wallet with about $50 to $100 AUSD, never your main wallet. I will not handle keys or funds; you set the environment variables (README). Be honest in the submission: it would have under two days of activity.	your call

D. While you work

- Keep the laptop awake and online: the data feed stopped twice while I worked (most likely the laptop sleeping or the network dropping; I did not confirm which). Keep the dashboard window (and for Envio, the Ubuntu indexer window) open while recording.

- Do not commit or share data/testnet/key.txt. It is a throwaway testnet key but treat it as a secret. It is git-ignored.
- If anything on the dashboard shows an error banner or 'Data' older than a few minutes, restart it with the command in the README; tell me and I will diagnose.

What to tell me when you have done something

Steps 3, 5 and 7 each unlock work I can do next: stable data (3), full-history indexer and the remaining checks (5), and the testnet bot run (7). If you give me nothing else, doing 1, 2 and 3 is enough for a defensible Track 1 submission that is honest about its limits.